Back To Practice

Q1

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

Completion Time (CT): The time at which a process finishes execution.

Turnaround Time (TAT): Total time taken from arrival to completion.

$TAT = CT - AT$

Waiting Time (WT): The time a process spends waiting in the ready queue.

$WT = TAT - BT$

put Format

e first input line contains an integer representing the number of ocesses.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int AT[n], BT[n], CT[n], TAT[n], WT[n];

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
        scanf("%d %d", &AT[i], &BT[i]);
    }

    CT[0] = AT[0] + BT[0];

    for(i = 1; i < n; i++) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

Completion Time (CT): The time at which a process finishes execution.

Turnaround Time (TAT): Total time taken from arrival to completion.

TAT = CT - AT$

Waiting Time (WT): The time a process spends waiting in the ready queue.

WT = TAT - BT$

Input Format

The first input line contains an integer representing the number of processes.

Select Language : C (GCC 9.2.0)

```c
    CT[0] = AT[0] + BT[0];

    for(i = 1; i < n; i++) {
        if(CT[i-1] < AT[i]) {
            CT[i] = AT[i] + BT[i];
        }
        else {
            CT[i] = CT[i-1] + BT[i];
        }
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

First-Come,First-Served CPU Scheduling Algorithm

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : C (GCC 9.2.0)

```c
        }
    }

    float totalTAT = 0, totalWT = 0;

    for(i = 0; i < n; i++) {
        TAT[i] = CT[i] - AT[i];
        WT[i] = TAT[i] - BT[i];

        totalTAT += TAT[i];
        totalWT += WT[i];
    }

    printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);
    printf("Average Waiting Time: %.2f\n", totalWT / n);

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1